

React Mastery Curriculum

From absolute React beginning to MERN / Next.js readiness

Scrum-style sprint plan based on the provided React PDFs plus requested beginner+ additions

Field	Plan
Audience	A learner who already knows core Vanilla JavaScript. The curriculum avoids re-teaching loops, arrays, DOM basics, classes, and promises except where React changes how they are used.
Delivery style	Each sprint has a goal, theory, diagrams, code, revision questions, and the required coding practice block.
Source boundary	Core sequence still follows the supplied PDFs. This revised version also incorporates the learner-requested beginner+ additions: useReducer, useContext/Context API, practical routing upgrades, production project structure, accessibility, React Testing Library, beginner TypeScript, Suspense/React.lazy, and basic React-backend integration.
Important assumption	The full Vanilla JavaScript curriculum text was not attached in this chat. I used the baseline stated in your prompt: you know core JS and need React-focused sequencing.

1. Audit Process and Source Inventory

I audited each PDF by extracting every lesson heading, then mapping each theory, technical warning, hook, state pattern, tooling note, and implementation detail into the sprint plan. The result is a React-first curriculum that avoids duplicate Vanilla JavaScript teaching and keeps every concept tied to a sprint.

Source PDFs audited

- React Fundamentals.pdf
- React state, Hooks and Routing.pdf
- Performance.pdf
- CSS Libraries and Framework.pdf
- Testing.pdf
- Basic React + backend integration bridge added for API services, environment variables, loading/error/data states, and basic auth flow concepts.
- Suspense and React.lazy() added lightly for route/component lazy loading, not advanced data Suspense.
- Beginner TypeScript with React added near the end through typed props and .tsx files.
- React Testing Library basics added to the testing sprint for component-level tests.
- Accessibility habits added early: semantic JSX, labels, buttons for actions, keyboard-friendly UI, alt text, and focus basics.
- Production project structure introduced early, after components and before larger projects.
- Practical React Router upgrades: normal routes, nested routes, dynamic route params, a 404 page, and a simple protected route.
- useContext and Context API near the hooks/state section, using createContext(), Provider, and useContext().
- useReducer basics after useState, for organized state logic without jumping to Redux.

This updated version keeps the original sprint structure and inserts the following beginner-appropriate concepts into the most relevant areas of the curriculum.

Revision add-ons requested by learner

Deep audit inventory

React Fundamentals.pdf

- Libraries vs frameworks: pre-built code, flexibility, opinionated structure, examples like jQuery, Angular, Remix, Next.js, and React as a UI library focused on the view layer.
- Single Page Applications: one HTML page, dynamic JavaScript updates, async data, accessibility issues for screen readers, navigation/history/bookmark issues, SEO issues, SSR/pre-rendering/SEO-friendly URLs, initial-load performance risk.
- What React is used for: reusable UI components, nested components, independent rendering, virtual DOM, and state-driven UI.

- Components and JSX: functional vs class components, modern functional components, JSX expressions, className, JSX comments, PascalCase component names, lowercase HTML tags, self-closing tags, parent wrapper requirement, Fragment and shorthand fragments.
- Default and named exports: .jsx files, App.jsx root component, one default export per file, multiple named exports, importing with braces, aliasing with as, mixed default/named imports.
- Vite setup: Node.js/npm, npm create vite command, npm install, package.json, dependencies, node_modules, npm run dev, localhost, Ctrl+C, editing src/App.jsx.
- Props: parent-to-child data, prop types, props object, destructuring, reusable components, children prop, component composition, spread props, immutable props, state for changing data.
- Conditional rendering: if statements, ternary operator, logical AND operator, truthy/falsy rendering behavior.
- Lists: map() in JSX, key prop, console warning, subtle UI bugs, index-key anti-pattern, stable unique IDs, UUID/crypto IDs, avoiding Math.random() during render, object list rendering.
- Inline styles: JavaScript style objects, double braces, camelCase CSS property names, dynamic styles from props or state.

React state, Hooks and Routing.pdf

- Events: Synthetic Event System, camelCase event names, function references, handle naming convention, event object, preventDefault, stopPropagation, inline arrow handlers with arguments.
- useState: dynamic component data, re-rendering, import patterns, state variable/setter/initial value, private isolated state, lifting state, top-level hook rule, multiple state values vs combined object state.
- Rendering: trigger, render, commit; virtual DOM; comparison with real DOM; only changed DOM nodes update.
- Object and array state: immutability, direct mutation bugs, setter functions, updater functions, spread copying, computed property names, avoid push/pop/splice, add with spread, remove with filter.
- Refs: useRef, ref attribute, current property, null before mount/after unmount, focusing inputs, DOM access best practices, checking ref.current.
- Effects: side effects, useEffect syntax, dependency array modes, document.title, cleanup for intervals/listeners/server connections, scroll listener cleanup.
- Custom hooks: reusable logic, use prefix, hooks folder convention, useDebounce with useState/useEffect, setTimeout/clearTimeout cleanup, search example.
- Forms: controlled and uncontrolled inputs, state vs refs, validation, disabling submit, when to use each, form best practices.
- React 19 hooks and server actions: useActionState syntax, state/action/isPending/actionFunction/initialState/permalink, server action forms, outside-form actions, startTransition for low-priority UI updates.
- Data fetching: Fetch API, Axios, SWR, loading/data/error state, async function inside useEffect, response checks, Axios JSON parsing, SWR fetcher, useFetch custom hook.
- useOptimistic: optimistic state, immediate feedback, pending task UI, async server action simulation, startTransition, delayed completion update.
- Memoization: why expensive re-renders happen, useMemo for computed values, useCallback for function references, React.memo for component memoization, dependencies.

- Dependency management: dependencies, dependency hell, npm, package.json, package-lock.json, node_modules, dependencies vs devDependencies, react/react-dom, npm update, yarn/pnpm.
- Routing: React Router, BrowserRouter, Routes, Route, path/element, index routes, nested routes, dynamic segments, useParams.
- React frameworks: Next.js and Remix, routing, image optimization, data fetching, authentication, route handlers, Next Image component.
- Prop drilling, state management, DevTools, React Server Components: single source of truth, Context API, Redux/Redux Toolkit/RTK Query, Zustand, Components/Profiler tabs, prop mismatch debugging, server/client component trade-offs and use client directive.

CSS Libraries and Framework.pdf

- CSS frameworks: why they help, consistency, browser compatibility, less repetition, responsive components, utility-first vs component-based systems.
- Tailwind CSS: granular utility classes and hover utilities.
- Bootstrap: pre-built component classes and HTML structure.
- Framework trade-offs: specificity issues, limited uniqueness, performance cost, choosing framework vs custom CSS vs a mix.
- CSS preprocessors: variables, nesting, modules, mixins, inheritance, operators, Sass/Less/Stylus, SCSS syntax, indented Sass syntax, @mixin/@include, compiled CSS, compile overhead, debugging compiled CSS.

Performance.pdf

- Real vs perceived performance: objective metrics vs user feeling, lazy loading, feedback, progress, avoiding layout jumps, font loading, interactive UI.
- Performance concepts: source order, progressive enhancement, critical rendering path, DOM, CSSOM, JS execution, render tree, layout, painting, latency, CDN, compression, image optimization.
- INP: interaction delay, why it matters, good/poor thresholds, DevTools measurement, long tasks, event handler optimization.
- Browser rendering: parse HTML, build DOM, process CSS into CSSOM, build render tree, layout/reflow, paint, compositing layers.
- Sustainability: energy use, carbon impact, file sizes, unnecessary scripts, minify, image formats, fewer requests, efficient algorithms, caching, CDN, Lighthouse, Website Carbon Calculator.
- Load-time optimization: compress media, WebP/AV1, lazy loading, fewer HTTP requests, inline critical CSS, sprites, cache-control, versioned assets, minify/compress, font-display.
- Time to usable and perceived features: load first content, defer non-critical JS, conditional CSS with media, skeleton screens, preloading, instant feedback.
- Metrics and tools: FCP, Speed Index, TBT, Bounce Rate, Unique Users, FCP vs INP, Chrome DevTools, Lighthouse, WebPageTest, PageSpeed Insights, RUM tools.
- Performance Web APIs: performance.now(), Performance Timing API, PerformanceObserver, DNS, TTFB, DOMContentLoaded, long tasks, layout shifts, FID.
- CSS/JS performance: remove unused CSS/JS, split files, simple selectors, cascade, preload, animation reflow, font limits, less JavaScript, built-in browser features, fewer animations, less DOM manipulation, code splitting, event listener cleanup, event delegation.

Testing.pdf

- Testing purpose: manual testing, automated testing, bugs, maintainability, developer job readiness.
- SDLC: planning, design, implementation, testing, deployment, maintenance, stakeholders, Waterfall, Agile, sprints.
- Unit testing: single responsibility, Jest, test files, npm script, expect/toBe, passing/failing output, empty input test, Jest/Mocha/Vitest, tests as documentation.
- TDD and BDD: red-green-refactor loop, debate around TDD, BDD tied to business goals, Gherkin Given/When/Then.
- Assertions: Chai, assert.isFunction, assert.equal, type/property/null assertions, should.js, expect.js, clear assert/failure messages.
- Mocks, stubs, fakes: mocked API responses, stubbed predefined values, fake in-memory DB/file system.
- Functional, E2E, usability, compatibility, performance, security, A/B, alpha/beta, regression testing: scenarios, tools, costs, and when each test type matters.

Coverage crosswalk

Sprint(s)	Main coverage	Source group
1	Libraries, frameworks, SPAs, React role, Vite setup	React Fundamentals
2	JSX, components, fragments, exports/imports, beginner project structure	React Fundamentals + Requested beginner+ additions
3	Props, children, composition, conditionals, lists, keys, inline styles, accessibility habits	React Fundamentals + Requested beginner+ additions
4	Events and forms	React state, Hooks and Routing
5	useState, useReducer basics, rendering, object/array immutability	React state, Hooks and Routing + Requested beginner+ additions
6	Refs, effects, custom hooks, debouncing, Context API basics	React state, Hooks and Routing + Requested beginner+ additions
7	Data fetching, basic backend integration bridge, useActionState, useOptimistic, startTransition	React state, Hooks and Routing + Requested beginner+ additions
8	Memoization, useMemo, useCallback, React.memo, dependency management	React state, Hooks and Routing
9	Routing, nested/dynamic/404/protected routes, Suspense/React.lazy, frameworks, Server Components, Next.js readiness	React state, Hooks and Routing + Requested beginner+ additions

10	Prop drilling, Context API, Redux/RTK, Zustand, DevTools debugging	React state, Hooks and Routing
11	CSS frameworks and preprocessors	CSS Libraries and Framework
12	Performance foundations, browser rendering, INP, sustainability	Performance
13	Performance optimization, metrics, tools, Web APIs, CSS/JS optimization	Performance
14	Testing foundations, unit testing, React Testing Library basics, TDD/BDD, assertions, mocks/stubs/fakes	Testing + Requested beginner+ additions
15	Functional, E2E, usability, compatibility, performance, security, A/B, alpha/beta, regression	Testing
16	Capstone integration across React, routing, state, fetching, testing, styling, performance, accessibility, and beginner TypeScript props	All PDFs + Requested beginner+ additions

2. Scrum-Style Curriculum Plan

Each sprint is written like a small Scrum learning cycle. Treat each sprint as one week if you study part-time, or two to three focused days if you study full-time. Do not skip the coding block. React becomes clear when you type it, break it, and fix it.

Learning flow

Foundations -> Components -> Project Structure -> Data Flow -> Accessible UI -> State -> useReducer -> Effects -> Context -> Server/Async -> Backend Bridge -> Optimization -> Practical Routing -> Lazy Loading -> Architecture -> Styling -> Performance -> Testing -> TypeScript Lite -> Capstone

Sprint 1: React Orientation, SPAs, and Vite Setup

Field	Plan
Sprint goal	Understand why React exists, what problem SPAs solve, and how to create a working project with Vite.
Definition of Done	You can explain the sprint concepts in simple words, build the sprint mini-project, fix the debug task, and answer the checkpoint questions without looking at notes.
Scrum rhythm	Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective

Sprint backlog

- Separate a library from a framework in your own words.
- Explain why React is called a UI library and why Next.js is called a framework.
- Describe SPA benefits and risks: accessibility, navigation, SEO, and initial loading.
- Create and run a Vite React project.

Theory in simple words

- A library gives you tools. You decide when to use them. A framework gives you a larger structure. It often decides the flow of the app.
- React focuses on the view layer. It helps you build UI with reusable pieces called components.
- A Single Page Application loads one HTML page. JavaScript changes the view without a full page refresh.
- SPAs can feel fast, but they need care. Screen readers may miss dynamic updates. Browser history and bookmarks need routing. Search engines may need SSR, pre-rendering, or SEO-friendly URLs. Initial load can be slow if the app bundle is large.
- Vite creates a small modern React setup. You install dependencies with `npm install` and run the development server with `npm run dev`.

Visual model

SPA idea

Traditional site: click link -> server sends new HTML page -> browser reloads

SPA: click link -> JS changes view in same page -> no full reload

React app boot flow

`index.html` -> `main.jsx` -> `App.jsx` -> components -> screen

Core code pattern

```
npm create vite@latest my-react-app -- --template react
cd my-react-app
```



```
npm install
npm run dev
# Open http://localhost:5173/
# Stop server with Ctrl + C
```

Revision checkpoint

1. Why is React more flexible than an opinionated framework?
2. Name three issues that SPAs can create if you do not handle them carefully.
3. What does npm install do in a Vite project?
4. What file do you usually edit first after opening a Vite React starter?

Mandatory Practice Coding Block

Warm-up drills:

- Create a folder name for a new React app.
- Write the Vite command from memory.
- Write one sentence that defines an SPA.

Guided coding task:

- Create a Vite React project, open src/App.jsx, replace the starter UI with one h1 and one p, then run it in the browser.

Debug this code:

Fix the command and explain each part.

Build something small:

- Build a one-screen “React Roadmap” page with a heading and three short cards: Components, State, Routing.

Challenge task:

- Add a short warning box listing two SPA risks and one way to reduce each risk.

Expected output / self-check:

- The app runs at localhost, App.jsx has been edited, and the page explains React without any full reload.

```
npm create vite my-react-app template react
cd my-react-app
npm install
npm dev
```

Sprint retrospective prompts

- What felt easy?
- What broke first?
- What concept needs one more tiny app?
- What would you refactor if this became a real project?

Sprint 2: JSX, Components, Fragments, and Module Exports

Field	Plan
Sprint goal	Build clean React components, organize them across files, and start a beginner-friendly production project structure.
Definition of Done	You can explain the sprint concepts in simple words, build the sprint mini-project, fix the debug task, and answer the checkpoint questions without looking at notes.
Scrum rhythm	Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective

Sprint backlog

- Write JSX with expressions inside braces.
- Use className, not class.
- Name custom components with a capital letter.
- Return multiple JSX elements using fragments.
- Export and import components with default and named exports.
- Decide what belongs in reusable components versus page-level components.
- Create a simple production-friendly src/ structure with components, pages, hooks, services, context, utils, and assets folders.

Theory in simple words

- A React component is like a function that returns UI.
- JSX looks like HTML, but it is JavaScript syntax. You place JavaScript expressions inside curly braces.
- class is a reserved JavaScript word, so JSX uses className.
- React treats lowercase tags like div as HTML elements. It treats capitalized tags like Greeting as custom components.
- JSX tags must close. If there are no children, use a self-closing tag like <Greeting />.
- A component can return only one parent value. Use a div, <Fragment>, or the shorthand <>...</> to group siblings.
- A file can have one default export. It can also have many named exports. Named exports are imported with curly braces.

Visual model

Component tree

```
<App />
|-- <Header />
|-- <Main />
```

```
| |-- <Card />
| |-- <Card />
|-- <Footer />
```

Core code pattern

```
// Greeting.jsx
export default function Greeting() {
  const name = "Maya";
  return (
    <>
      <h1 className="title">Hello {name}</h1>
      <p>Welcome to React.</p>
    </>
  );
}
```

```
// App.jsx
import Greeting from "./Greeting";

export default function App() {
  return <Greeting />;
}
```

- Keep shared providers in context/ only when several branches of the app need the same value.
- Keep browser/API communication in services/ so components stay focused on UI.
- Keep reusable UI in components/. Keep full screens or route pages in pages/.

```
src/
components/ # reusable UI pieces: Button, Card, Navbar
pages/      # route-level screens: Home, Products, Login
hooks/      # custom hooks: useFetch, useDebounce
services/   # API helpers: api.js, authService.js
context/    # Context providers: AuthContext, ThemeContext
utils/      # pure helper functions: formatPrice, countDone
assets/     # images, icons, static files
```

As projects grow, do not keep every file directly inside src/. Start with a small structure that separates reusable UI, pages, hooks, service calls, shared context, utilities, and assets.

Beginner production project structure

Revision checkpoint

5. Why must component names usually start with a capital letter?
6. What problem do fragments solve?
7. How do you import a named export?
8. Why does JSX use className?
9. Which folder should hold reusable components, and which folder should hold route-level pages?

Mandatory Practice Coding Block

Warm-up drills:

- Write a JSX h1 that shows a name variable.
- Write one self-closing component tag.
- Write a fragment with two paragraphs.

Guided coding task:

- Create Header.jsx, ProfileCard.jsx, and App.jsx. Export Header as default and ProfileCard as a named export. Render both in App.

Debug this code:

Fix the JSX errors and export/import mistakes.

Build something small:

- Build a personal intro page with Header, AboutBox, and SkillBadge components.

Challenge task:

- Create an Animals.jsx file with one default component and two named components. Import all three into App.

Expected output / self-check:

- No adjacent JSX error appears, all components render, and imports use the correct default/named syntax.

```
function App() {  
  return <h1>Hello</h1>  
  <p>React</p>  
}  
  
function profilecard() {  
  return <div class="card">Card</div>  
}  
  
import { Header } from "./Header";
```

- Write a one-sentence rule for what belongs in services/ and utils/.
- Create the recommended src/ folders and move at least one component into components/ and one screen into pages/.

Beginner+ practice extension

Sprint retrospective prompts

- What felt easy?
- What broke first?
- What concept needs one more tiny app?
- What would you refactor if this became a real project?

Sprint 3: Props, Composition, Conditions, Lists, Keys, and Inline Styles

Field	Plan
Sprint goal	Make components reusable, data-driven, and accessible by default.
Definition of Done	You can explain the sprint concepts in simple words, build the sprint mini-project, fix the debug task, and answer the checkpoint questions without looking at notes.
Scrum rhythm	Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective

Sprint backlog

- Pass props from parent to child.
- Destructure props for readability.
- Use children for wrapper components.
- Spread an object into props.
- Render conditions with if, ternary, and &&.
- Render arrays with map and stable keys.
- Use inline style objects and dynamic styles.
- Keep interactive elements keyboard-friendly and avoid divs pretending to be buttons.
- Write accessible JSX with semantic elements, alt text, labels, and real buttons.

Theory in simple words

- Props are data passed from a parent component to a child component. Props can be strings, numbers, booleans, objects, or arrays.
- Props are immutable. A child should not change them. Use state when data must change.
- children is a special prop. It contains whatever JSX you place between a component opening tag and closing tag.
- The spread operator can pass many object properties as individual props.
- Conditional rendering means showing different UI depending on data. Use if for bigger branches, ternary for inline either/or, and && for show-or-hide.
- map() turns an array into an array of JSX elements. Every list item needs a stable key so React can track changes.
- Avoid index keys when lists can reorder, sort, or filter. Avoid Math.random() keys because they change on every render.
- Inline styles are JavaScript objects. CSS names become camelCase, such as backgroundColor and fontSize.

Visual model

Data flow with props

Parent owns data

|

v

<ProfileCard name="Ana" role="Frontend" />

|

v

Child reads props and returns UI

Props go down. Events can tell parent what happened.

Core code pattern

```
function Card({ children }) {
  return <section style={{ border: "1px solid #ccc", padding: "12px" }}>{children}</section>;
}

function UserCard({ id, name, online }) {
  return (
    <Card>
      <h2>{name}</h2>
      {online ? <p>Online</p> : <p>Offline</p>}
      {id && <small>ID: {id}</small>}
    </Card>
  );
}

export default function App() {
  const users = [
    { id: "u1", name: "Ana", online: true },
    { id: "u2", name: "Sam", online: false },
  ];
  return users.map((user) => <UserCard key={user.id} {...user} />);
}
```

Beginner rule: if the user can click it, ask whether keyboard users can also reach and activate it.

```
function EmailField() {
  return (
    <form>
      <label htmlFor="email">Email</label>
      <input id="email" name="email" type="email" />
      <button type="submit">Join</button>
      
    </form>
  );
}
```

- Keep focus visible and predictable, especially when opening or closing UI.

- Give meaningful alt text to images that communicate content. Use empty alt text only for decorative images.
- Use a real button for actions. Avoid clickable divs unless you fully handle keyboard and ARIA behavior.
- Connect labels to inputs with htmlFor and id.
- Use semantic JSX: header, main, nav, section, article, form, label, button, and footer when they match the meaning.

Accessibility is not an advanced add-on. It is a set of small habits you practice while writing JSX.

Accessibility habits in React

Revision checkpoint

10. Why are props useful for reusable components?
11. What does the children prop contain?
12. When is using an array index as a key risky?
13. What are the two pairs of curly braces in style={{ color: "red" }} doing?
14. Why is a real button better than a clickable div for most actions?

Mandatory Practice Coding Block

Warm-up drills:

- Write a Greeting component that accepts a name prop.
- Write a Card component that renders children.
- Write a list of three items with stable keys.

Guided coding task:

- Build ProductCard and ProductList. ProductList owns an array of products. ProductCard receives title, price, and inStock. Show “Sold out” only when needed.

Debug this code:

Fix the bad prop access, bad key, and incorrect style syntax.

Build something small:

- Build a mini “Team Directory” that renders a list of people with online/offline labels.

Challenge task:

- Add a wrapper Card component using children and dynamic border color based on a featured prop.

Expected output / self-check:

- Each item renders once, no key warning appears, and online/offline/featured UI changes from data only.

```
function User(props) {
  return <h2>{name}</h2>;
}
```

```
const users = [{ name: "Ana" }, { name: "Sam" }];
```

```
function App() {
```

```
return users.map((user) => <User key={Math.random()} user={user} style="color: red" />);  
}
```

- Replace any non-semantic clickable element with a button where appropriate.
- Audit your Team Directory for labels, alt text, semantic elements, and clickable divs.

Beginner+ practice extension

Sprint retrospective prompts

- What felt easy?
- What broke first?
- What concept needs one more tiny app?
- What would you refactor if this became a real project?

Sprint 4: React Events and Form Inputs

Field	Plan
Sprint goal	Handle user actions in the React way and control form data safely.
Definition of Done	You can explain the sprint concepts in simple words, build the sprint mini-project, fix the debug task, and answer the checkpoint questions without looking at notes.
Scrum rhythm	Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective

Sprint backlog

- Use onClick, onChange, and onSubmit in camelCase.
- Pass function references instead of strings.
- Use event.preventDefault and event.stopPropagation when needed.
- Pass extra data with inline arrow handlers.
- Build controlled and uncontrolled inputs.
- Validate inputs and show feedback.

Theory in simple words

- React wraps native browser events in a Synthetic Event. This helps events behave consistently across browsers.
- React event names use camelCase: onClick, onSubmit, onChange.
- Give the handler function itself to the JSX attribute. Do not call it immediately unless you wrap it in another function.
- event.preventDefault() stops default browser actions like form reload.
- event.stopPropagation() stops an event from bubbling to parent elements.
- Controlled inputs store the value in React state and update it with onChange. This is best for validation and dynamic UI.
- Uncontrolled inputs keep their value in the DOM. Use refs when you only need the value on submit or when integrating non-React code.
- Forms should prevent default submission, validate data, and show clear loading/error/success feedback.

Visual model

Controlled input cycle

User types -> onChange runs -> setState(new value) -> React re-renders -> input shows state value

```
<input value={name} onChange={...} />
```

Core code pattern

```
import { useState, useRef } from "react";

export default function Signup() {
  const [name, setName] = useState("");
  const noteRef = useRef(null);

  function handleSubmit(event) {
    event.preventDefault();
    const note = noteRef.current ? noteRef.current.value : "";
    console.log({ name, note });
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={name} onChange={(e) => setName(e.target.value)} placeholder="Name" />
      <textarea ref={noteRef} placeholder="Optional note" />
      <button disabled={!name.trim()}>Submit</button>
    </form>
  );
}
```

Revision checkpoint

15. Why should `onClick` receive `handleClick` instead of `handleClick()`?
16. What does `preventDefault` do in a form?
17. When would an uncontrolled input be reasonable?
18. What is one benefit of controlled inputs?

Mandatory Practice Coding Block

Warm-up drills:

- Write a button with `onClick`.
- Write an `onSubmit` handler that prevents refresh.
- Write an input controlled by state.

Guided coding task:

- Build a `FeedbackForm` with `name`, `rating`, and `message`. Disable submit until `name` and `message` are not empty. Log the data on submit.

Debug this code:

Fix the event handler and controlled input mistakes.

Build something small:

- Build a small “Contact Us” form with live character count.

Challenge task:

- Add an uncontrolled hidden “ref code” field with `useRef` and include it in the submit data.

Expected output / self-check:

- Typing updates the screen, submitting does not reload, validation works, and disabled state is clear.

```
function App() {  
  const [name, setName] = useState("");  
  function submit(e) {  
    console.log(name);  
  }  
  return <form onsubmit={submit()}>  
    <input value={name} onChange={setName(e.target.value)} />  
  </form>;  
}
```

Sprint retrospective prompts

- What felt easy?
- What broke first?
- What concept needs one more tiny app?
- What would you refactor if this became a real project?

Sprint 5: useState, Rendering, and Immutable State Updates

Field	Plan
Sprint goal	Understand React re-rendering, update state without mutation bugs, and learn beginner useReducer for organized state logic.
Definition of Done	You can explain the sprint concepts in simple words, build the sprint mini-project, fix the debug task, and answer the checkpoint questions without looking at notes.
Scrum rhythm	Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective

Sprint backlog

- Use useState for dynamic UI.
- Explain trigger, render, and commit.
- Lift state when siblings must share data.
- Update object state with updater functions and spread.
- Update array state with spread, map, and filter.
- Avoid direct mutation of state.
- Define actions and return a new state from a reducer without mutation.
- Use useReducer for simple organized state when useState logic starts branching.

Theory in simple words

- State is data that can change over time and control what the component shows.
- Each component instance has its own private state. Rendering the same component twice creates separate state.
- Hooks must be called at the top level of a component, not inside loops, conditions, or nested functions.
- When state or props change, React triggers an update. It renders the component again in memory, then commits the changed DOM parts to the page.
- The virtual DOM lets React compare old UI and new UI before touching the real DOM.
- Objects and arrays in state must be treated as immutable. Create a new object or array instead of editing the old one.
- For object forms, use input name attributes and computed property names: [name]: value.
- For arrays, add with spread, remove with filter, and update with map.

Visual model

React render cycle

State/props change

|
v
Trigger update
|
v
Render component functions again
|
v
Compare virtual DOM with previous version
|
v
Commit only needed real DOM changes

Core code pattern

```
import { useState } from "react";

export default function ProfileEditor() {
  const [user, setUser] = useState({ name: "Lina", age: 20, city: "Colombo" });
  const [skills, setSkills] = useState(["JS", "CSS"]);

  function handleChange(e) {
    const { name, value } = e.target;
    setUser((prev) => ({ ...prev, [name]: value }));
  }

  function addSkill(skill) {
    setSkills((prev) => [...prev, skill]);
  }

  function removeSkill(skill) {
    setSkills((prev) => prev.filter((item) => item !== skill));
  }

  return <div>{user.name} has {skills.length} skills.</div>;
}
```

Do not replace every `useState` with `useReducer`. Use it when the state rules become easier to read as actions.

- The reducer returns a new state object.
- `action.type` describes what happened.
- `dispatch` sends an action to the reducer.
- `state` is the current data.

```
import { useReducer } from "react";
```

```
const initialState = { count: 0 };
```

```

function reducer(state, action) {
  if (action.type === "increment") {
    return { count: state.count + 1 };
  }

  if (action.type === "decrement") {
    return { count: state.count - 1 };
  }

  if (action.type === "reset") {
    return initialState;
  }

  return state;
}

export default function CounterReducer() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <>
      <p>{state.count}</p>
      <button onClick={() => dispatch({ type: "decrement" })}>-</button>
      <button onClick={() => dispatch({ type: "increment" })}>+</button>
      <button onClick={() => dispatch({ type: "reset" })}>Reset</button>
    </>
  );
}

```

A reducer is a function that receives the current state and an action. It returns the next state. It must not mutate the old state.

useState is still your first tool. Use useReducer when one component has state logic with several action types, such as increment, decrement, reset, add item, remove item, or update field.

useReducer basics after useState

Revision checkpoint

19. What are the three rendering stages in React?
20. Why does direct mutation often fail to update the UI?
21. Why can setting { age: value } replace the whole user object?
22. Which array method is useful for removing an item without mutation?
23. When is useReducer clearer than several related useState calls?

Mandatory Practice Coding Block

Warm-up drills:

- Write a counter with useState.

- Write an object state update using spread.
- Write an array add operation using spread.

Guided coding task:

- Build a ProfileEditor with name, age, and city inputs. Use one handleChange function. Add a Skills list with add/remove buttons.

Debug this code:

Fix direct object and array mutation bugs.

Build something small:

- Build a small “Packing List” app with add and remove.

Challenge task:

- Add a checkbox to mark each item packed using map to update one item.

Expected output / self-check:

- Every update changes the UI immediately, existing object fields are not lost, and no array mutation methods are used.

```
function List() {
  const [items, setItems] = useState(["A"]);
  function addItem() {
    items.push("B");
    setItems(items);
  }
  return <button onClick={addItem}>{items.length}</button>;
}
```

- Explain why the reducer returns new state instead of editing the old state.
- Rebuild the counter once with useReducer and add increment, decrement, and reset actions.

Beginner+ practice extension

Sprint retrospective prompts

- What felt easy?
- What broke first?
- What concept needs one more tiny app?
- What would you refactor if this became a real project?

Sprint 6: Refs, Effects, Cleanup, and Custom Hooks

Field	Plan
Sprint goal	Use refs for DOM access, effects for side effects without leaks, and Context API for simple shared app state.
Definition of Done	You can explain the sprint concepts in simple words, build the sprint mini-project, fix the debug task, and answer the checkpoint questions without looking at notes.
Scrum rhythm	Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective

Sprint backlog

- Create refs with useRef.
- Attach refs to DOM elements.
- Check ref.current before using it.
- Use useEffect with no dependency array, empty array, and reactive dependencies.
- Return cleanup functions for timers and event listeners.
- Extract reusable logic into a custom hook.
- Use Context for app-wide values like an auth user or theme, not for every small local state value.
- Create a simple context with createContext(), a Provider component, and useContext().

Theory in simple words

- Refs let you access a DOM element directly. They can hold mutable values, but state is better for UI data.
- A ref object has a current property. It starts as null, points to the element after mount, and returns to null after unmount.
- Effects are for work outside rendering: fetching data, changing document.title, local storage, geolocation, timers, subscriptions, and event listeners.
- With dependencies omitted, useEffect runs after every render. With [], it runs once after first render. With [value], it runs after first render and when that value changes.
- Cleanup prevents old timers, listeners, or connections from staying alive after they are no longer needed.
- A custom hook is a reusable function that starts with use. It can use built-in hooks inside it.
- Debouncing waits until a user stops typing for a short time before running logic. useDebounce combines state, effects, setTimeout, and clearTimeout.

Visual model

Effect with cleanup

Render -> effect starts timer/listener
Update -> cleanup old timer/listener -> run new effect
Unmount -> cleanup final timer/listener

This prevents duplicate work and memory leaks.

Core code pattern

```
import { useEffect, useRef, useState } from "react";

function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return debounced;
}

export default function SearchBox() {
  const inputRef = useRef(null);
  const [query, setQuery] = useState("");
  const debouncedQuery = useDebounce(query, 500);

  useEffect(() => {
    document.title = debouncedQuery ? `Search: ${debouncedQuery}` : "Search";
  }, [debouncedQuery]);

  return <input ref={inputRef} value={query} onChange={(e) => setQuery(e.target.value)} />;
}
```

Beginner warning: Context is not a replacement for all state. Keep local state local when only one component needs it.

- `useContext()` reads the value from inside a child component.
- `Provider` makes a value available below it in the tree.
- `createContext()` creates the shared context object.

```
import { createContext, useContext, useState } from "react";
```

```
const AuthContext = createContext(null);
```

```
export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
```

```
  function login(name) {
    setUser({ name });
  }
}
```

```

function logout() {
  setUser(null);
}

return (
  <AuthContext.Provider value={{ user, login, logout }}>
    {children}
  </AuthContext.Provider>
);
}

export function useAuth() {
  return useContext(AuthContext);
}

function Navbar() {
  const { user, logout } = useAuth();
  return user ? (
    <button onClick={logout}>Logout {user.name}</button>
  ) : (
    <p>Please log in</p>
  );
}

```

Component tree goal

```

App
  AuthProvider
    Navbar -> reads user from context
    Dashboard

```

Context lets a component read shared data without passing the same prop through every level. For beginners, use it for values like current user, theme, or language.

Context API beginner primer

Revision checkpoint

24. When should you use a ref instead of state?
25. What does an empty dependency array mean?
26. Why should event listeners be removed in cleanup?
27. Why must a custom hook name start with use?
28. What are the three basic pieces of Context API: createContext, Provider, and useContext?

Mandatory Practice Coding Block

Warm-up drills:

- Write useRef(null).
- Write an effect that sets document.title.

- Write a cleanup function that clears a timeout.

Guided coding task:

- Build a SearchBox that focuses the input with a button and shows a debounced query below it.

Debug this code:

Fix missing dependencies and missing cleanup.

Build something small:

- Build a small “Delayed Preview” app: type text, preview updates after 700ms.

Challenge task:

- Add a scroll listener that shows “You scrolled” and clean it up properly.

Expected output / self-check:

- No timer duplicates occur, the input focus button works, and document.title updates only after the debounce delay.

```
useEffect(() => {
  const id = setInterval(() => console.log(query), 1000);
}, []);

function focusInput() {
  inputRef.current.focus();
}
```

- Keep at least one unrelated local state value outside Context to prove you are not overusing it.
- Build an AuthProvider with user, login, and logout, then let Navbar read user with useContext.

Beginner+ practice extension

Sprint retrospective prompts

- What felt easy?
- What broke first?
- What concept needs one more tiny app?
- What would you refactor if this became a real project?

Sprint 7: Data Fetching, Server Actions, useActionState, and useOptimistic

Field	Plan
Sprint goal	Fetch data safely, create responsive async UI, and organize beginner API/backend integration patterns.
Definition of Done	You can explain the sprint concepts in simple words, build the sprint mini-project, fix the debug task, and answer the checkpoint questions without looking at notes.
Scrum rhythm	Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective

Sprint backlog

- Fetch data with loading, data, and error states.
- Use async/await inside useEffect by creating an inner async function.
- Compare Fetch API, Axios, and SWR at a high level.
- Extract data fetching into a useFetch hook.
- Understand server actions and useActionState.
- Use startTransition when action updates should be low priority.
- Use useOptimistic for instant UI feedback while async work completes.
- Use environment variables for API base URLs and describe a beginner auth flow concept.
- Create a services/api.js file for reusable API calls.

Theory in simple words

- React does not force one data-fetching method. You can use Fetch API, Axios, SWR, or a custom hook.
- A common fetch component needs three states: loading, data, and error.
- Do not make the useEffect callback itself async. Put an async function inside the effect and call it.
- Fetch needs res.json(). Axios parses JSON for you. SWR gives shorter syntax by using a fetcher function and returning data/error.
- useActionState updates state based on an action result. It returns state, an action function, and isPending.
- Server actions run on the server. In frameworks that support them, forms can use actions without a separate API endpoint.
- startTransition marks an update as lower priority so React can keep the UI responsive.
- useOptimistic shows the expected result immediately, then reconciles with the real result later.

Visual model

Async UI states

Start fetch/action

- |-- loading/isPending true -> show feedback
- |-- success -> show data/result
- |-- error -> show error message

Optimistic update:

User submits -> show new item now -> server confirms later -> finalize item

Core code pattern

```
import { useEffect, useState, useOptimistic, startTransition } from "react";
```

```
function useFetch(url) {
  const [status, setStatus] = useState({ data: null, loading: true, error: null });
  useEffect(() => {
    async function load() {
      try {
        const res = await fetch(url);
        if (!res.ok) throw new Error("Network error");
        setStatus({ data: await res.json(), loading: false, error: null });
      } catch (error) {
        setStatus({ data: null, loading: false, error });
      }
    }
    load();
  }, [url]);
  return status;
}
```

```
function TaskList({ tasks, addTask }) {
  const [optimisticTasks, addOptimisticTask] = useOptimistic(
    tasks,
    (state, text) => [...state, { text, pending: true }]
  );
```

```
  function handleSubmit(e) {
    e.preventDefault();
    const text = new FormData(e.currentTarget).get("task");
    startTransition(() => addOptimisticTask(text));
    addTask(text);
    e.currentTarget.reset();
  }
}
```

Security details belong in the later MERN/Next.js curriculum. For now, learn the flow and keep the API code organized.

Basic auth flow concept

Login form -> POST /login -> backend verifies user

-> frontend stores user/auth state

-> protected routes check that state

-> logout clears the state

```
// .env
```

```
VITE_API_BASE_URL=https://api.example.com
```

```
// src/services/api.js
```

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL;
```

```
export async function getPosts() {  
  const res = await fetch(`${API_BASE_URL}/posts`);  
  if (!res.ok) throw new Error("Failed to load posts");  
  return res.json();  
}
```

```
// src/pages/PostsPage.jsx
```

```
useEffect(() => {  
  async function loadPosts() {  
    try {  
      setLoading(true);  
      setPosts(await getPosts());  
    } catch (error) {  
      setError(error);  
    } finally {  
      setLoading(false);  
    }  
  }  
}
```

```
  loadPosts();  
}, []);
```

- Understand the basic auth flow before building advanced JWT or refresh-token logic.
- Keep loading, error, and data states in the UI so the user is never stuck with a blank screen.
- Use environment variables for API base URLs instead of hardcoding them in every component.
- Put API helper functions in services/api.js or services/authService.js.

This is not a full MERN sprint yet. It is a beginner bridge so your React components do not directly scatter fetch calls everywhere.

Basic React + backend integration bridge

Revision checkpoint

29. Why do we keep loading and error state when fetching?
30. Why should useEffect not be directly marked async?
31. What does isPending mean in useActionState?

- 32. How is optimistic UI different from a normal loading spinner?
- 33. Why is a services/api.js file better than scattering fetch calls through many components?

Mandatory Practice Coding Block

Warm-up drills:

- Write loading/data/error state.
- Write a fetch inside useEffect.
- Write useOptimistic syntax from memory.

Guided coding task:

- Build a Posts viewer using Fetch API. Then refactor the fetch logic into useFetch.

Debug this code:

Fix the async effect and missing error handling.

Build something small:

- Build a “Quick Tasks” app that adds a pending item immediately before the async save finishes.

Challenge task:

- Add a simulated 2-second save and show “Saving...” next to pending tasks.

Expected output / self-check:

- Loading, success, and error UI all work; optimistic tasks appear instantly and settle after save.

```
useEffect(async () => {  
  const res = await fetch("/api/posts");  
  const data = await res.json();  
  setData(data);  
}, []);
```

- Draw the login -> user state -> protected route -> logout flow in your notes.
- Move one fetch function into services/api.js and read the base URL from `import.meta.env.VITE_API_BASE_URL`.

Beginner+ practice extension

Sprint retrospective prompts

- What felt easy?
- What broke first?
- What concept needs one more tiny app?
- What would you refactor if this became a real project?

Sprint 8: Memoization and Dependency Management

Field	Plan
Sprint goal	Avoid unnecessary work and manage project packages safely.
Definition of Done	You can explain the sprint concepts in simple words, build the sprint mini-project, fix the debug task, and answer the checkpoint questions without looking at notes.
Scrum rhythm	Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective

Sprint backlog

- Explain memoization in simple words.
- Use `useMemo` for expensive computed values.
- Use `useCallback` for function references.
- Use `React.memo` to avoid unnecessary child re-renders.
- Explain `package.json`, `package-lock.json`, `node_modules`, `dependencies`, and `devDependencies`.
- Install, update, and choose packages with care.

Theory in simple words

- Memoization means remembering a result so React does not repeat expensive work when the inputs are the same.
- `useMemo` caches a computed value. It should depend on the values used inside the calculation.
- `useCallback` caches a function reference. This helps when a function is a dependency or is passed to a memoized child.
- `React.memo` wraps a component and skips re-rendering when its props have not changed.
- Memoization is not magic. Use it for real expensive work, large lists, or child components that re-render too often.
- A dependency is a package or module your app needs. Poorly managed dependencies can cause dependency hell.
- `package.json` lists metadata, scripts, dependencies, and `devDependencies`. `package-lock.json` locks exact versions. `node_modules` stores installed package code.
- `react` and `react-dom` are core React dependencies. Jest-like tools belong in `devDependencies` because they are only needed during development/testing.
- `npm` is one package manager. `yarn` and `pnpm` are alternatives. Existing teams usually decide the package manager for you.

Visual model

Memoization decision

Does this render feel slow?

|

no -> do not optimize yet

|

yes

|

Is the same calculation/function repeated with same inputs?

|

yes -> useMemo/useCallback/React.memo carefully

Core code pattern

```
import React, { useCallback, useMemo, useState } from "react";

const ChildButton = React.memo(function ChildButton({ onAdd }) {
  console.log("ChildButton rendered");
  return <button onClick={onAdd}>Add</button>;
});

export default function ExpensiveCounter() {
  const [count, setCount] = useState(0);
  const [tick, setTick] = useState(0);

  const square = useMemo(() => {
    console.log("Calculating square");
    return count * count;
  }, [count]);

  const add = useCallback(() => {
    setCount((prev) => prev + 1);
  }, []);

  return <><p>{square}</p><ChildButton onAdd={add} /><button onClick={() => setTick(tick + 1)}>Tick</button></>;
}
```

Revision checkpoint

34. What does useMemo remember?
35. What does useCallback remember?
36. Why might React.memo not help if a function prop changes every render?
37. What is the difference between dependencies and devDependencies?

Mandatory Practice Coding Block

Warm-up drills:

- Write useMemo with a dependency array.
- Write useCallback with an updater function.

- Identify dependencies vs devDependencies in package.json.

Guided coding task:

- Build an ExpensiveSquare component and prove it only recalculates when num changes.

Debug this code:

Fix the missing dependency and unnecessary child re-render.

Build something small:

- Build a mini “Cart Summary” that calculates total price with useMemo.

Challenge task:

- Wrap a child CheckoutButton with React.memo and keep its handler stable with useCallback.

Expected output / self-check:

- Console logs prove expensive work does not rerun on unrelated state changes, and package roles are explained correctly.

```
const total = useMemo(() => cart.reduce((sum, item) => sum + item.price, 0), []);
```

```
const checkout = () => {
  setSubmitted(true);
};
```

```
const Button = React.memo(({ onClick }) => <button onClick={onClick}>Pay</button>);
```

Sprint retrospective prompts

- What felt easy?
- What broke first?
- What concept needs one more tiny app?
- What would you refactor if this became a real project?

Sprint 9: Routing, React Frameworks, and Server Components

Field	Plan
Sprint goal	Move from one-screen React apps to multi-view apps with practical routing, simple protected routes, lazy-loaded components, and a beginner view of frameworks like Next.js.
Definition of Done	You can explain the sprint concepts in simple words, build the sprint mini-project, fix the debug task, and answer the checkpoint questions without looking at notes.

Sprint backlog

- Install and set up React Router.
- Wrap the app with BrowserRouter.
- Create Routes and Route elements.
- Use index routes, nested routes, and dynamic segments.
- Read route params with useParams.
- Explain what React frameworks add beyond React.
- Explain client components vs React Server Components.
- Lazy-load a route component with React.lazy() and Suspense.
- Add a 404 route and a simple protected route.

Theory in simple words

- React SPAs need routing when the app has multiple views. React Router is a third-party library for this.
- BrowserRouter enables browser history support. Routes holds route definitions. Route connects a path to UI through the element prop.
- An index route is the default view for a route segment.
- Nested routes append child paths to parent paths.
- Dynamic segments start with a colon, such as :productId. useParams reads the value from the URL.
- A React framework can add routing, image optimization, data fetching, authentication, route handlers, redirects, and more.
- Next.js includes file-system routing and route handlers such as GET or POST inside app/api/route.js.
- The Next Image component extends img and can improve loading by serving correct sizes and lazy-loading images.
- React Server Components render on the server and send final HTML to the client. They can fetch server data directly and reduce client JavaScript.
- Server Components cannot use hooks, Web APIs, or browser event listeners. Interactive components need the "use client" directive.

Visual model

Route tree example

```
/
|-- index -> <Home />
|-- about -> <About />
|-- products
    |-- index -> <ProductsHome />
    |-- trending -> <Trending />
    |-- :category -> <Category />
    |-- :category/:productId -> <ProductDetail />
```

Core code pattern

```
import { BrowserRouter, Routes, Route, useParams } from "react-router";

function Category() {
  const { category } = useParams();
  return <h1>Category: {category}</h1>;
}

export default function AppRoutes() {
  return (
    <BrowserRouter>
      <Routes>
        <Route index element={<h1>Home</h1>} />
        <Route path="about" element={<h1>About</h1>} />
        <Route path="products/:category" element={<Category />} />
      </Routes>
    </BrowserRouter>
  );
}

// Server component idea: async function can fetch before HTML is sent.
async function UsersServer() {
  const res = await fetch("https://jsonplaceholder.typicode.com/users");
  const users = await res.json();
  return users.map((user) => <p key={user.id}>{user.name}</p>);
}
```

Beginner warning: learn Suspense for component lazy loading first. Do not go deep into Suspense for data fetching yet.

```
import React, { Suspense } from "react";

const Dashboard = React.lazy(() => import("./pages/Dashboard"));

function App() {
  return (
    <Suspense fallback={<p>Loading page...</p>}>
      <Dashboard />
    </Suspense>
  );
}
```

React.lazy() lets you load a component only when it is needed. Suspense shows a fallback while the lazy component loads. Use this lightly for large pages or route-based code splitting.

Suspense and React.lazy() intro

```
import { BrowserRouter, Routes, Route, Navigate } from "react-router";
```

```
function ProtectedRoute({ isLoggedIn, children }) {  
  if (!isLoggedIn) {  
    return <Navigate to="/login" replace />;  
  }  
}
```

```
  return children;  
}
```

```
export default function AppRoutes() {  
  const isLoggedIn = false;
```

```
  return (  
    <BrowserRouter>  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/login" element={<Login />} />  
        <Route path="/courses/:courseId" element={<CourseDetail />} />  
        <Route  
          path="/dashboard"  
          element={(  
            <ProtectedRoute isLoggedIn={isLoggedIn}>  
              <Dashboard />  
            </ProtectedRoute>  
          )}  
        />  
        <Route path="*" element={<NotFound />} />  
      </Routes>  
    </BrowserRouter>  
  );  
}
```

- A simple protected route blocks private screens when the user is not logged in.
- A 404 route catches unknown URLs.
- Dynamic route params read values from the URL, such as :productId.
- Nested routes group child pages under a parent route, such as /courses/:courseId/lessons.
- Normal routes show top-level pages like /, /about, and /login.

A beginner app should go beyond one basic route. Add the routing patterns you will use in real projects.

Practical React Router upgrades

Revision checkpoint

38. What does BrowserRouter do?

39. How do nested routes form a URL?

40. What does useParams return?

41. Why can a Server Component fetch data without `useEffect`?

42. What should happen when a user opens an unknown route or a protected route while logged out?

Mandatory Practice Coding Block

Warm-up drills:

- Write one Route with path and element.
- Write a dynamic route with `:id`.
- Write `useParams` inside a component.

Guided coding task:

- Build a small product catalog with Home, Products, Category, and ProductDetail routes.

Debug this code:

Fix the incorrect import, wrong path, and missing `BrowserRouter` wrapper.

Build something small:

- Build a mini “Course Browser” with route `/courses/:courseId`.

Challenge task:

- Add nested routes under `/courses` for overview and lessons.

Expected output / self-check:

- Direct URLs work, dynamic params display, and route structure is easy to explain.

```
import { Routes, Route, useParams } from "react-router";
```

```
function Product() {  
  const { id } = useParams();  
  return <h1>{id}</h1>;  
}
```

```
export default function App() {  
  return <Routes><Route path="/product/:id" component={<Product />} /></Routes>;  
}
```

- Add one `ProtectedRoute` wrapper and one lazy-loaded page with `Suspense` fallback.
- Add a `NotFound` page with `path="**"`.

Beginner+ practice extension

Sprint retrospective prompts

- What felt easy?
- What broke first?
- What concept needs one more tiny app?
- What would you refactor if this became a real project?

Sprint 10: Prop Drilling, State Management, and React DevTools

Field	Plan
Sprint goal	Recognize when local state is not enough and debug React apps with the right tools.
Definition of Done	You can explain the sprint concepts in simple words, build the sprint mini-project, fix the debug task, and answer the checkpoint questions without looking at notes.
Scrum rhythm	Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective

Sprint backlog

- Explain prop drilling and why it becomes hard to scale.
- Use single source of truth intentionally.
- Use Context API for simple shared state.
- Compare Context API, Redux/Redux Toolkit, RTK Query, and Zustand.
- Inspect components with React DevTools Components tab.
- Profile unnecessary re-renders with Profiler.
- Debug a prop mismatch.

Theory in simple words

- Prop drilling means passing data through components that do not use it, just to reach a deep child.
- A single source of truth means keeping shared data in one clear owner component or store.
- Context API is built into React. It is good for data like theme or user authentication status.
- Context can cause unnecessary re-renders when frequently changing values are shared widely.
- Redux uses a central store, actions, reducers, and middleware. It is useful in large apps where updates must be predictable.
- Redux Toolkit reduces Redux boilerplate. RTK Query helps with server state and data fetching patterns.
- Zustand gives a lighter hook-based store. It can be easier for small to medium apps and does not require a Provider for basic use.
- React DevTools adds Components and Profiler tabs. Components shows props, state, context, component tree, DOM links, and source links. Profiler helps find unnecessary re-renders and commit duration issues.
- A prop mismatch happens when a parent passes one prop name but the child expects another. DevTools helps trace the prop path.

Visual model

Prop drilling vs Context

Prop drilling:

App -> Parent -> Child -> Grandchild
(Parent/Child may not use the data)

Context:

Provider stores shared value

|-- Any component can read it with useContext

Core code pattern

```
import { createContext, useContext, useState } from "react";

const CounterContext = createContext(null);

function CounterProvider({ children }) {
  const [count, setCount] = useState(0);
  return (
    <CounterContext.Provider value={{ count, setCount }}>
      {children}
    </CounterContext.Provider>
  );
}

function Counter() {
  const { count, setCount } = useContext(CounterContext);
  return <button onClick={() => setCount((c) => c + 1)}>Count: {count}</button>;
}
```

Revision checkpoint

43. What makes prop drilling hard to maintain?
44. When is Context API a good choice?
45. Why does Redux need actions and reducers?
46. Which DevTools tab helps you inspect props and state?

Mandatory Practice Coding Block

Warm-up drills:

- Draw a three-level prop drilling tree.
- Create a Context object.
- Use useContext to read count.

Guided coding task:

- Refactor a drilled greeting prop into Context. Then inspect it with React DevTools.

Debug this code:

Fix the prop mismatch in the nested component chain.

Build something small:

- Build a “Theme Toggle” with Context and useContext.

Challenge task:

- Add a frequently changing counter and observe re-renders in Profiler. Decide whether Context is still a good fit.

Expected output / self-check:

- Theme state is shared without drilling, prop mismatch is fixed, and DevTools inspection is documented.

```
function Child({ response }) {  
  return <Grandchild reply={response} />;  
}
```

```
function Grandchild({ response }) {  
  return <h2>{response}</h2>;  
}
```

Sprint retrospective prompts

- What felt easy?
- What broke first?
- What concept needs one more tiny app?
- What would you refactor if this became a real project?

Sprint 11: Styling Systems: CSS Frameworks and Preprocessors

Field	Plan
Sprint goal	Choose styling tools for React projects without ignoring trade-offs.
Definition of Done	You can explain the sprint concepts in simple words, build the sprint mini-project, fix the debug task, and answer the checkpoint questions without looking at notes.
Scrum rhythm	Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective

Sprint backlog

- Explain what a CSS framework is.
- Compare utility-first and component-based frameworks.
- Identify Tailwind-like utility classes and Bootstrap-like component classes.
- Explain framework advantages and disadvantages.
- Explain preprocessors and Sass features.
- Write SCSS variables and mixins.
- Understand compile overhead and debugging trade-offs.

Theory in simple words

- A CSS framework gives pre-written CSS so you can style faster and keep a consistent design.
- Utility-first frameworks use small classes for one job each, such as margin, padding, text color, or hover background.
- Component-based frameworks give larger ready-made UI patterns such as cards, list groups, modals, and navbars.
- Frameworks help speed, consistency, cross-browser design, and responsive layouts. They can also create specificity conflicts, make websites look less unique, and add performance cost if large.
- A CSS preprocessor adds extra syntax to CSS and compiles it into normal CSS that browsers understand.
- Sass supports variables, nesting, modules, mixins, inheritance, and operators.
- SCSS uses braces and semicolons like CSS. Indented Sass uses indentation and no semicolons/braces.
- Mixins group reusable style declarations. @mixin defines them and @include uses them.
- Preprocessors can add a compile step and make debugging harder because the browser sees compiled CSS, not the original source syntax.

Visual model

Styling choice

Need speed and consistent UI? -> CSS framework
Need tiny reusable utility classes? -> utility-first
Need ready-made components? -> component-based
Need scalable custom CSS logic? -> preprocessor
Need full uniqueness and tiny CSS? -> custom CSS carefully

Core code pattern

```
/* Utility-first style idea */
<button className="bg-blue-500 text-white px-4 py-2 rounded-full">
  Save
</button>

/* SCSS */
$primary-color: #3498eb;

@mixin center-flex {
  display: flex;
  justify-content: center;
  align-items: center;
}

.card {
  @include center-flex;
  background-color: $primary-color;
}
```

Revision checkpoint

47. What is the difference between utility-first and component-based CSS frameworks?
48. Name two disadvantages of CSS frameworks.
49. What does Sass compile into?
50. What do @mixin and @include do?

Mandatory Practice Coding Block

Warm-up drills:

- Write three utility classes you would expect in a utility-first framework.
- Write an SCSS variable.
- Write a tiny mixin and include it.

Guided coding task:

- Style your previous Course Browser using either utility-style class naming or custom SCSS. Add one reusable card style.

Debug this code:

Fix the Sass mixin syntax and React className usage.

Build something small:

- Build a small “Pricing Cards” layout with consistent button styles.

Challenge task:

- Create a theme variables file and use it in two components.

Expected output / self-check:

- The styling choice is justified, the UI is consistent, and no obvious specificity fight is introduced.

```
$primary-color #3498eb;
```

```
@mixin center-flex  
  display: flex;  
  justify-content: center;
```

```
<section class="card">Hello</section>
```

Sprint retrospective prompts

- What felt easy?
- What broke first?
- What concept needs one more tiny app?
- What would you refactor if this became a real project?

Sprint 12: Performance Foundations: Real Speed, Perceived Speed, Browser Rendering, and INP

Field	Plan
Sprint goal	Understand what browsers do and why users feel an app is fast or slow.
Definition of Done	You can explain the sprint concepts in simple words, build the sprint mini-project, fix the debug task, and answer the checkpoint questions without looking at notes.
Scrum rhythm	Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective

Sprint backlog

- Separate real performance from perceived performance.
- Explain source order and progressive enhancement.
- Explain the critical rendering path.
- Explain browser rendering: DOM, CSSOM, render tree, layout, paint.
- Explain latency and how CDNs/compression help.
- Define INP and measure it with DevTools.
- Connect performance with sustainability.

Theory in simple words

- Real performance is measured with objective numbers: load time, server response time, rendering time, and similar metrics.
- Perceived performance is how fast and reliable the site feels to users.
- Lazy loading, loading indicators, progress updates, reserved image space, similar fallback fonts, and responsive UI controls can improve how fast the app feels.
- Source order matters. Put important content higher in HTML and defer non-essential scripts.
- Progressive enhancement means the core page should work with HTML first, then improve with CSS and JavaScript.
- The critical rendering path is the browser path from code to pixels: parse HTML, parse CSS, run JavaScript, build render tree, layout, paint.
- Latency is the travel time between browser and server. CDNs, compression, image optimization, and lazy loading can reduce the feeling of delay.
- INP measures the delay from user interaction to the next visual update. Good INP is generally below 200ms, and poor INP is above 500ms.
- Performance also affects sustainability. Smaller files, fewer requests, efficient algorithms, caching, and CDNs reduce wasted processing and energy.

Visual model

Browser rendering path

HTML -> DOM
CSS -> CSSOM
DOM + CSSOM -> Render Tree -> Layout/Reflow -> Paint -> Composite -> Screen

Slow JS or render-blocking CSS can delay this path.

Core code pattern

```
<!-- Good source order idea -->
<h1>Welcome to FastSite</h1>
<p>Important content appears early.</p>
<script src="analytics.js" defer></script>

<!-- Reserve image space to reduce layout jump -->

```

Revision checkpoint

51. What is the difference between real and perceived performance?
52. List the main steps of the critical rendering path.
53. What does INP measure?
54. Why can optimized performance reduce energy use?

Mandatory Practice Coding Block

Warm-up drills:

- Draw DOM -> CSSOM -> render tree.
- Write an image tag with width, height, alt, and loading="lazy".
- Write one example of defer.

Guided coding task:

- Open one of your React apps in Chrome DevTools Performance tab. Record an interaction and identify whether it feels delayed.

Debug this code:

Fix the layout shift and render-blocking choices.

Build something small:

- Build a small “Performance Notes” page with early text, lazy image, and a feedback button.

Challenge task:

- Add a heavy loop button, record the delay, then remove or reduce the work and compare feel.

Expected output / self-check:

- The page avoids obvious layout jumps, key content appears early, and you can explain INP in one sentence.

```

```

```
<script src="large-non-critical.js"></script>  
<button onclick="doVeryHeavyWork()">Search</button>
```

Sprint retrospective prompts

- What felt easy?
- What broke first?
- What concept needs one more tiny app?
- What would you refactor if this became a real project?

Sprint 13: Performance Optimization, Metrics, Tools, and Web APIs

Field	Plan
Sprint goal	Measure and improve loading, interaction, CSS, and JavaScript performance.
Definition of Done	You can explain the sprint concepts in simple words, build the sprint mini-project, fix the debug task, and answer the checkpoint questions without looking at notes.
Scrum rhythm	Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective

Sprint backlog

- Reduce page loading time with media optimization, fewer requests, caching, compression, and font strategy.
- Improve time to usable with critical content, lazy loading, defer, and conditional CSS.
- Use skeleton screens, preloading, and instant feedback for perceived performance.
- Track FCP, Speed Index, TBT, Bounce Rate, Unique Users, and INP.
- Use Chrome DevTools, Lighthouse, WebPageTest, PageSpeed Insights, and RUM tools.
- Use performance.now, Performance Timing API, and PerformanceObserver.
- Apply CSS and JavaScript performance techniques.

Theory in simple words

- Optimize images and videos by compressing, using modern formats like WebP/AV1, and lazy loading off-screen media.
- Reduce HTTP requests with combined files, inline critical CSS, and CSS sprites where appropriate.
- Use Cache-Control and versioned assets so returning visitors load faster while still getting updates.
- Minify CSS/JS and enable compression such as Gzip.
- Limit font variants and use font-display to reduce font loading delay.
- Time to usable improves when essential content loads first, non-critical JS uses defer, and non-critical CSS loads conditionally with media attributes.
- Skeleton screens can feel better than spinners because they show page shape immediately.
- FCP measures first visible content. INP measures feedback after interaction. TBT measures blocked main-thread time. Speed Index measures how fast visible parts paint.
- DevTools and Lighthouse are fast local checks. WebPageTest tests locations/devices. PageSpeed Insights gives improvement suggestions. RUM tools show real user behavior.
- performance.now measures precise code timing. Performance Timing API breaks down page loading. PerformanceObserver can watch long tasks, layout shifts, and interaction-related entries.
- CSS performance: remove unused CSS, split files, use media for non-critical CSS, minify/compress, use simple selectors, rely on cascade, preload critical assets, reduce reflow-heavy animations, limit fonts.

- JavaScript performance: use less JS, remove unused code, prefer built-in browser features, reduce DOM work, split critical/non-critical modules, preload/defer carefully, minify, remove unused listeners, and use event delegation.

Visual model

Metric cheat sheet

FCP -> first content appears after load

INP -> feedback after click/type

TBT -> main thread blocked by heavy JS

Speed Index -> visible page paints over time

Bounce Rate -> users leave without interacting

Unique Users -> individual visitors

Core code pattern

```
const start = performance.now();
// run code to measure
const end = performance.now();
console.log(`Execution time: ${end - start}ms`);

const observer = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    console.log("Long task:", entry.duration);
  }
});
observer.observe({ type: "longtask", buffered: true });

<link rel="stylesheet" href="print.css" media="print" />
<link rel="preload" href="/next-page" as="document" />
```

Revision checkpoint

55. Which metric is about first visible content?
56. Which tool gives lab-style audits inside Chrome DevTools?
57. What can PerformanceObserver track?
58. Why should old event listeners be removed?

Mandatory Practice Coding Block

Warm-up drills:

- Write performance.now timing code.
- Write a lazy iframe.
- Write a conditional stylesheet with media.

Guided coding task:

- Run Lighthouse on your React app. Record FCP, Speed Index, TBT, and one recommendation. Apply one safe improvement.

Debug this code:

Fix the performance anti-patterns in CSS and JS.

Build something small:

- Build a mini performance dashboard that measures one function with `performance.now` and shows the result.

Challenge task:

- Add a `PerformanceObserver` for long tasks and create a button that demonstrates a long task.

Expected output / self-check:

- You have before/after metrics or notes, one improvement applied, and a short explanation of why it helped.

```
document.querySelectorAll("li").forEach((li) => {  
  li.addEventListener("click", handleClick);  
});  
  
.card .body .content ul li span strong {  
  color: red;  
}  
  
setInterval(() => console.log("still running"), 1000);
```

Sprint retrospective prompts

- What felt easy?
- What broke first?
- What concept needs one more tiny app?
- What would you refactor if this became a real project?

Sprint 14: Testing Foundations: SDLC, Unit Tests, TDD/BDD, Assertions, Mocks, Stubs, and Fakes

Field	Plan
Sprint goal	Learn why tests exist and write small reliable unit tests plus beginner React component tests.
Definition of Done	You can explain the sprint concepts in simple words, build the sprint mini-project, fix the debug task, and answer the checkpoint questions without looking at notes.
Scrum rhythm	Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective

Sprint backlog

- Explain manual vs automated testing.
- Explain SDLC stages and how testing fits.
- Write a unit test with Jest-style syntax.
- Use expect and matchers.
- Understand passing and failing test output.
- Explain TDD and BDD.
- Write assertions with clear messages.
- Compare mocks, stubs, and fakes.
- Query rendered UI with screen and assert what the user sees.
- Test React components with React Testing Library.

Theory in simple words

- Manual testing means a person clicks through the app. It matters, but it can miss hidden bugs in complex apps.
- Automated testing means test code checks expected behavior without manual clicking.
- SDLC stages are planning, design, implementation, testing, deployment, and maintenance. Agile breaks work into sprints. Waterfall finishes one phase before the next.
- Unit testing checks a small function or component in isolation. It works well with the single responsibility principle.
- Jest uses test(), expect(), and matchers such as toBe(). A failing test shows expected and received values.
- Tests can document expected behavior, not just catch bugs.
- TDD means write a test, watch it fail, write code, watch it pass, refactor, repeat.
- BDD aligns tests with business goals and uses scenario language like Given, When, Then.
- Assertions check that code behaves as expected. Clear assert messages and clear failure messages make debugging easier.
- Mocks replace real dependencies with controlled data. Stubs return predefined values. Fakes are simple working versions of real systems, such as an in-memory database.

Visual model

TDD loop

Write test -> Run test -> Fails -> Write minimum code -> Passes -> Refactor -> Repeat

Testing pyramid idea

Many unit tests

Some integration/functional tests

Few expensive E2E tests

Core code pattern

```
// formatName.js
export function formatName(str = "") {
  if (!str) return "";
  return str.charAt(0).toUpperCase() + str.slice(1);
}
```

```
// formatName.test.js
import { formatName } from "./formatName.js";
```

```
test("capitalizes the first letter", () => {
  expect(formatName("react")).toBe("React");
});
```

```
test("returns empty string for no input", () => {
  expect(formatName()).toBe("");
});
```

Beginner goal: write one component test that proves a button, form label, or conditional message appears correctly.

- Do not start with complex API mocking. First test simple props, conditional text, buttons, and forms.
- Prefer queries that match what users see, such as text, labels, roles, and placeholders.
- screen queries the rendered output.
- render() mounts the component in a test environment.

```
import { render, screen } from "@testing-library/react";
import Button from "./Button";
```

```
test("shows the button label", () => {
  render(<Button label="Submit" onClick={() => {}} />);

  expect(screen.getByText("Submit")).toBeInTheDocument();
});
```

React Testing Library helps you test components the way users experience them. Instead of testing internal implementation, you render the component and look for visible UI.

React Testing Library basics

Revision checkpoint

- 59. Why is manual testing alone not enough for complex apps?
- 60. What are the SDLC stages?
- 61. What is the TDD cycle?
- 62. What is the difference between a mock, a stub, and a fake?
- 63. What does React Testing Library encourage you to test: implementation details or user-visible behavior?

Mandatory Practice Coding Block

Warm-up drills:

- Write one `test()` block.
- Write one `expect(...).toBe(...)`.
- Write Given/When/Then for login.

Guided coding task:

- Create a `formatName` function and tests for lowercase input, empty input, and already-capitalized input.

Debug this code:

Fix the failing test and improve the function.

Build something small:

- Build a tiny test file for a cart total utility used by your React app.

Challenge task:

- Mock an API response array and test a function that extracts titles from it.

Expected output / self-check:

- At least three tests pass, one intentional failing test was understood, and the mock/stub/fake difference is written in your notes.

```
export function formatName(str) {  
  return "This is incorrect";  
}  
  
test("capitalizes", () => {  
  expect(formatName("hello")).toBe("Hello");  
});
```

- Use `screen.getByText`, `screen.getByRole`, or `screen.getByLabelText` at least once.
- Write one React Testing Library test for a Button, Alert, or simple Form component.

Beginner+ practice extension

Sprint retrospective prompts

- What felt easy?
- What broke first?
- What concept needs one more tiny app?
- What would you refactor if this became a real project?

Sprint 15: Testing Strategy: Functional, E2E, Usability, Compatibility, Performance, Security, A/B, Alpha/Beta, and Regression

Field	Plan
Sprint goal	Select the right test type for the right risk.
Definition of Done	You can explain the sprint concepts in simple words, build the sprint mini-project, fix the debug task, and answer the checkpoint questions without looking at notes.
Scrum rhythm	Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective

Sprint backlog

- Differentiate functional and non-functional testing.
- Explain smoke tests and login scenarios.
- Use E2E testing for critical user flows.
- Explain usability testing types and tools.
- Explain compatibility testing across browsers, OS, hardware, networks, and mobile.
- Explain load, stress, soak, spike, and breakpoint testing.
- Explain security testing principles and threats.
- Explain A/B, alpha, beta, and regression testing.

Theory in simple words

- Functional testing checks whether features work as expected. Unit testing is part of functional testing. Smoke testing is a basic check before deeper tests.
- Non-functional testing checks qualities such as performance and reliability.
- E2E tests simulate real user flows like registration, login, checkout, or donation. They are useful but expensive to set up and maintain.
- Playwright, Cypress, Selenium, and Puppeteer are common E2E/regression tool choices.
- Usability testing uses real users to find design, UX, and functionality issues. Types include explorative, comparative, assessment, and validation testing. Tools include Loop11, Maze, Userbrain, UserTesting, and UXTweak.
- Compatibility testing checks devices, OS, browser differences, hardware, networks, backward/forward compatibility, and mobile behavior.
- Performance testing checks speed, responsiveness, scalability, and stability. Load tests normal/peak load. Stress tests extreme load. Soak tests long duration. Spike tests sharp load changes. Breakpoint/capacity testing increases load until degradation.
- Security testing checks confidentiality, integrity, authentication, authorization, availability, and non-repudiation. Threats include XSS, SQL injection, and DoS. Tools fall into SAST, DAST, and penetration testing.

- A/B testing compares two versions and uses data, such as click-through rate, to choose the better one. Tools include GrowthBook and LaunchDarkly.
- Alpha testing uses selected testers before release. Beta testing uses real users. Regression testing reruns tests after changes to catch unintended breakage. Retesting checks known fixes; regression looks for unknown breakage from recent changes.

Visual model

Test choice map

Small pure logic -> unit test
 Feature behavior -> functional test
 Critical user journey -> E2E test
 Design confusion -> usability test
 Browser/device risk -> compatibility test
 Speed/load risk -> performance test
 Security risk -> security test
 Recent change risk -> regression test
 Experiment -> A/B test

Core code pattern

```
// Playwright-style E2E shape
import { test, expect } from "@playwright/test";

test("user can open products page", async ({ page }) => {
  await page.goto("/products");
  await expect(page.getByRole("heading", { name: "Products" })).toBeVisible();
});

// BDD scenario shape
// Given the user is logged in
// When the user adds a product to cart
// Then the cart count should increase
```

Revision checkpoint

64. What is the difference between functional and non-functional testing?
65. Why are E2E tests usually reserved for critical flows?
66. Name three security testing principles.
67. What is the difference between retesting and regression testing?

Mandatory Practice Coding Block

Warm-up drills:

- Write one smoke test idea.
- Write one E2E scenario.

- Write one compatibility checklist item.

Guided coding task:

- Create a test strategy for your Course Browser: list unit tests, functional tests, one E2E flow, compatibility checks, and regression triggers.

Debug this code:

Fix the test type labels in the broken strategy.

Build something small:

- Build a one-page “QA Plan” component that shows test categories for a React app.

Challenge task:

- Add a fake A/B test plan for a CTA button and define the success metric.

Expected output / self-check:

- Every test type is matched to a realistic risk, and E2E is not overused for simple pure functions.

Strategy:

- Use E2E to test `formatPrice()` returns "\$4.00".
- Use unit testing to test full checkout across browser and payment page.
- Use A/B testing to find SQL injection issues.
- Use usability testing to measure server load under 10,000 users.

Sprint retrospective prompts

- What felt easy?
- What broke first?
- What concept needs one more tiny app?
- What would you refactor if this became a real project?

Sprint 16: Capstone Sprint: React Learning Dashboard

Field	Plan
Sprint goal	Combine the React concepts into one small but realistic app, add a light TypeScript props layer, and prepare for MERN/Next.js learning.
Definition of Done	You can explain the sprint concepts in simple words, build the sprint mini-project, fix the debug task, and answer the checkpoint questions without looking at notes.
Scrum rhythm	Plan -> Learn -> Build -> Debug -> Review -> Refactor -> Retrospective

Sprint backlog

- Build a Vite React app with reusable components.
- Use props, children, conditionals, lists, keys, and inline or framework-style styling.
- Use state, immutable object/array updates, forms, refs, effects, and a custom hook.
- Fetch data with loading/error states or a useFetch hook.
- Use routing with at least one dynamic segment.
- Avoid prop drilling with Context where it makes sense.
- Use DevTools and a simple performance check.
- Write unit tests and a test strategy for higher-level testing.
- Understand when a React file becomes .tsx and how prop types help prevent mistakes.
- Type one or two components with beginner TypeScript props.

Theory in simple words

- The capstone should not be big. It should be complete. Small complete apps teach better than huge unfinished apps.
- Use React for UI structure and local state. Use Context only where shared state makes sense.
- Use a custom hook when logic is reused or distracts from the component UI.
- Use routing to simulate real app views. Dynamic routes prepare you for full-stack app routing later.
- Use loading, error, empty, and success states. This makes the UI robust.
- Use DevTools Components to check props/state and Profiler to notice unexpected re-renders.
- Use performance thinking from the start: lazy media, clear feedback, low layout shift, and no unnecessary heavy JS.
- Testing is part of the build, not something after everything else. Unit tests protect utilities. E2E ideas protect critical flows.
- MERN/Next.js readiness means you can now understand where Express/MongoDB/API routes or Next.js Server Components would fit, but those topics need separate resources beyond these PDFs.

Visual model

Capstone architecture

```
<App>
  <BrowserRouter>
    <LearningProvider>
      /      -> Dashboard
      /topics    -> TopicList
      /topics/:topic -> TopicDetail
      /practice   -> PracticePlanner
    </LearningProvider>
  </BrowserRouter>
```

Shared state: completed topics

Custom hook: useFetch or useDebounce

Testing: utilities + strategy for routes/forms

Core code pattern

```
// Suggested data shape
const topics = [
  { id: "jsx", title: "JSX", sprint: 2, done: false },
  { id: "state", title: "State", sprint: 5, done: false },
  { id: "routing", title: "Routing", sprint: 9, done: false },
];

// Utility to test
export function countDone(items) {
  return items.filter((item) => item.done).length;
}
```

Capstone option: convert only Button.tsx and ProductCard.tsx first. Keep the rest JavaScript until the TypeScript syntax feels natural.

```
type ProductCardProps = {
  title: string;
  price: number;
  inStock: boolean;
};

function ProductCard({ title, price, inStock }: ProductCardProps) {
  return (
    <article>
      <h2>{title}</h2>
      <p>${price}</p>
      {inStock ? <button>Buy</button> : <p>Sold out</p>}
    </article>
```

```

    );
  }

  type ButtonProps = {
    label: string;
    onClick: () => void;
  };

  function Button({ label, onClick }: ButtonProps) {
    return <button onClick={onClick}>{label}</button>;
  }

  export default Button;

```

- Use clear types for event handlers only after you understand the JavaScript version.
- Start by typing props. Do not try to type every advanced pattern at once.
- Use .tsx for React components written with TypeScript.

Do not start React with TypeScript on day one. First become comfortable with JavaScript React. Near the end, add TypeScript lightly so props mistakes become easier to catch.

TypeScript with React beginner layer

Revision checkpoint

68. Which capstone state should stay local and which should be shared?
69. Where does a custom hook improve the code?
70. Which route should be dynamic and why?
71. Which unit tests and E2E tests protect the most important behavior?
72. What is the simplest useful TypeScript type to add to a React component?

Mandatory Practice Coding Block

Warm-up drills:

- Sketch the route tree.
- Write the topic data array.
- Write one utility test for countDone.

Guided coding task:

- Build the capstone in steps: layout, components, routes, topic list, topic detail, completion state, practice form, custom hook, loading/error UI, test utility.

Debug this code:

Fix the capstone architecture bug: prop drilling everything through every route and mutating topic objects directly.

Build something small:

- Build the “React Learning Dashboard” with Dashboard, Topics, TopicDetail, and PracticePlanner pages.

Challenge task:

- Add a simulated server action idea: mark a task optimistic first, then finalize it after a delay. Add one Lighthouse improvement.

Expected output / self-check:

- The app runs, routes work, state updates immutably, shared state is not overused, one custom hook exists, one unit test passes, and performance/testing notes are written.

```
// Buggy idea to refactor
```

```
function markDone(id) {  
  const topic = topics.find((t) => t.id === id);  
  topic.done = true;  
  setTopics(topics);  
}
```

```
<Route path="topics/:topicId" element={<TopicDetail topics={topics} setTopics={setTopics} />} />
```

- Do not convert the full app yet; the goal is confidence, not migration speed.
- Convert one small component to .tsx and add a props type.

Beginner+ practice extension

Sprint retrospective prompts

- What felt easy?
- What broke first?
- What concept needs one more tiny app?
- What would you refactor if this became a real project?

3. Final Mastery Checklist

- I can explain React as a UI library and explain what frameworks add.
- I can create a Vite React app and understand package.json, node_modules, and dependencies.
- I can write JSX with expressions, className, fragments, and valid component names.
- I can organize components with default and named exports.
- I can pass props, use children, spread props, and respect prop immutability.
- I can render conditionals and lists with stable keys.
- I can handle React events, forms, preventDefault, stopPropagation, controlled inputs, and uncontrolled refs.
- I can use useState and update objects/arrays immutably.
- I can explain trigger, render, commit, virtual DOM, and real DOM updates.
- I can use refs safely and effects with correct dependencies and cleanup.
- I can write a custom hook such as useDebounce or useFetch.
- I can fetch data with Fetch API, understand Axios/SWR patterns, and render loading/error/success states.
- I can explain useActionState, server actions, isPending, startTransition, and useOptimistic at a beginner-friendly level.
- I can optimize expensive computations and child re-renders with useMemo, useCallback, and React.memo when needed.
- I can create routes with React Router, dynamic segments, nested routes, and useParams.
- I can explain Next.js-style framework features and React Server Component limits.
- I can avoid prop drilling with Context or choose a state library when the app grows.
- I can inspect React apps with React DevTools Components and Profiler.
- I can compare CSS frameworks, preprocessors, Sass syntax, mixins, and trade-offs.
- I can explain real/perceived performance, critical rendering path, browser rendering, INP, FCP, TBT, Speed Index, and sustainability.
- I can run basic performance measurements and apply CSS/JS performance techniques.
- I can write unit tests, explain TDD/BDD, use assertions, and understand mocks/stubs/fakes.
- I can choose test types for functional, E2E, usability, compatibility, performance, security, A/B, alpha/beta, and regression risks.
- I can type a simple React component props object in TypeScript and understand why .tsx is used.
- I can write a beginner React Testing Library test that checks user-visible output.
- I can keep API calls in a services file, use environment variables for base URLs, and explain a basic auth flow concept.
- I can lazy-load a page or component with React.lazy() and Suspense.
- I can build practical routes: normal routes, nested routes, dynamic params, a 404 route, and a simple protected route.
- I can create and use a simple Context API setup with createContext(), Provider, and useContext().
- I can use useReducer for simple action-based state without mutating the current state.
- I can build accessible JSX with semantic elements, connected labels, real buttons, keyboard-friendly interaction, meaningful alt text, and basic focus awareness.
- I can organize a beginner React app with components, pages, hooks, services, context, utils, and assets folders.

4. Suggested Portfolio Evidence

- One Vite React component gallery using props, children, lists, and conditions.
- One state-heavy mini app that updates objects and arrays immutably.
- One routed mini app with dynamic route parameters.
- One data-fetching app with loading, error, empty, and success UI.
- One performance note showing before/after Lighthouse or DevTools observations.
- One testing note with unit tests and a higher-level test strategy.
- One capstone React Learning Dashboard that ties the entire curriculum together.
- One API services file using an environment variable for the base URL.
- One typed Button.tsx or ProductCard.tsx component using beginner TypeScript props.
- One React Testing Library component test.
- One accessibility checklist applied to a form or component gallery.
- One routed mini app with nested routes, dynamic params, a 404 page, a protected route, and one lazy-loaded route.
- One Context demo for theme or auth state using createContext, Provider, and useContext.
- One small useReducer demo with at least three actions.

5. Next Learning Boundary

After this curriculum, the next step toward MERN and Next.js is not more React basics. You will already have beginner bridges for Context, reducers, practical routing, accessibility, React Testing Library, TypeScript props, lazy loading, and API service organization. The next separate learning track should cover MongoDB, Express, Node.js APIs, secure authentication, deployment, and a full Next.js app-router project in depth.